



INF2010 – Structures de données et algorithmes

Hiver 2026

TP No. 4

Groupe 2

2435049 – Nicolas Mihali

2420324 – Akhan Mehmet Sozen

Date de la remise : 22 Mars 2026

1. Décrivez la logique de votre solution et analysez la complexité temporelle et spatiale.

Description de la logique

La solution se fait sur trois étapes :

En premier, on parcourt la chaîne text compter le nombre de fois qu'un caractère y apparaît. En effet, `text.toCharArray()` converti la chaîne en un tableau de caractères puis la boucle visite ainsi chaque caractère un par un. Pour chacun des caractères, `map.getOrDefault(c, 0)` récupère le compteur actuel de c dans la map, ou 0 si c n'y est pas encore. On ajoute 1 au compteur, puis `map.put(c, ...)` enregistre la nouvelle valeur. À la fin, la map contient la fréquence de chaque lettre distincte de text.

Ensuite, on construit un `ArrayList<LetterFrequency>` à partir des entrées de la map. On encapsule chaque paire (caractère, fréquence) dans un objet `LetterFrequency` et ensuite, on appelle `Heap.heapify(heap, Comparator.reverseOrder())` pour en faire un max heap selon la fréquence, cela va permettre au caractère le plus fréquent de se retrouver à la racine.

Enfin, on procède à l'extraction de $k + 1$ éléments successifs avec

`Heap.pop(heap, Comparator.reverseOrder())`. L'élément extrait va être le k-ième caractère le plus fréquent et ce dernier sera retourné.

Analyse de la complexité temporelle

Soit n : longueur de text, m : nombre de lettres distinctes dans text et k : paramètre d'entrée.

- Le parcours de text pour remplir la HashMap se fait en $O(n)$. Chaque opération put et getOrDefault est $O(1)$ en moyenne.
- La construction de l'ArrayList à partir des m entrées de la map s'effectue en $O(m)$.
- L'appel à heapify sur le tableau de la taille m s'effectue en $O(m)$
- La boucle où on fait l'extraction appelle pop $k + 1$ fois. Chaque appel a pop se fait en $O(\log m)$, donc la boucle est $O((k + 1)\log m) = O(k * \log m)$

La complexité temporelle est donc de $O(n + m + m + k * \log m) = O(n + m + k * \log m)$. Mais comme m est inférieur ou égal à n , on a donc $O(n + k * \log m)$. On peut avoir $O(n)$ si n est supérieur à $k * \log m$ et $O(k * \log m)$ si n est inférieur.

Analyse de la complexité spatiale

- La HashMap stocke un nombre maximal de m entrées, donc $O(m)$.
- L'ArrayList du heap stocke m éléments, donc $O(m)$.

La complexité spatiale est donc de $O(m + m) = O(m)$.

2. Comment avez-vous validé votre code? Pouvez-vous être certains de la validité de votre algorithme?

La validation du code a été effectuée à l'aide d'un SIAG. On lui a soumis notre implémentation et lui avons demandé de vérifier la logique de notre solution. Le SIAG a ainsi validé que notre algorithme retourne le bon élément.

Il n'est cependant pas possible d'être sûr de la validité de notre algorithme seulement avec cette méthode, car le SIAG peut lui-même se tromper et ne couvrir pas tous les cas possibles.

3. Décrivez *brièvement* votre utilisation des SIAG dans ce TP

On a utilisé un SIAG comme outil d'aide à la réflexion et à la validation. Le SIAG a aidé à structurer l'approche pour pouvoir implémenter l'algorithme. Il a aussi aidé à la vérification du code. Aucune ressource a été utilisée autre que les notes de cours ainsi qu'un SIAG.

En conclusion, le SIAG a été efficace pour arriver rapidement à une solution fonctionnelle, mais se fier uniquement à lui pour la validation pourrait entraîner des risques, car celui-ci peut se tromper aussi.